

0.99. Une lettre qui tire — le A bouge, le tir garde sa ligne

Le 0.98, et le A bouge avec la croix. Le tir part de là où est le A ; sa ligne, `ty`, est recopiée une fois au départ : si le A s'en va, le tir continue tout droit.

1. Ce qu'il faut comprendre

C'est le 0.98, avec une chose en plus : le A bouge. La croix le déplace, un pas toutes les 150 ms, exactement comme le O du 0.93.1 : on calcule la case d'arrivée `nx`, `ny`, on regarde si elle est dans l'écran et vide, puis on y va.

Dans l'écran : `nx < 20 && ny < 17`. Et à gauche ? En colonne 0, `ax - 1` ne donne pas -1 : un `uint8_t` fait le tour et donne **255** (le 0.95). 255 n'est pas plus petit que 20 : sortir à gauche est refusé par le même test. Pareil en haut.

Ce qui est nouveau ici : le tir a SA ligne, `ty`. Au 0.98, le tir avançait sur la ligne `ay`, celle du A. Mais si le A monte pendant que le tir vole, le tir monterait avec lui ! Un tir ne fait pas ça : une fois parti, il va **tout droit**.

La solution : recopier. Au départ du tir, `ty = ay` : `ty` prend la valeur que `ay` a **à ce moment-là**. Ensuite, le tir n'utilise plus que `ty`. Le A peut bouger, `ay` peut changer : `ty`, lui, ne change pas.

`ax < 19` : en colonne 19, il n'y a plus de case à droite du A. On ne lance pas.

Le A ne marche pas sur son tir : il ne va que sur une case **vide** (`lire(nx, ny) == 0`), et le « . » n'est pas vide.

Déroulons : A en (2, 8). BAS, BAS, BAS : A en (2, 11). Bouton A : `tx = 3`, `ty = 11`. HAUT : le A remonte en (2, 10), `ay = 10` ; mais `ty` vaut toujours 11 : le tir continue sur la ligne 11.

Essaie : remplace `ty` par `ay` dans le bloc qui fait avancer le tir, et bouge pendant qu'il vole : le tir suit le A comme une ombre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **716 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

```
Gravé dans la cartouche :
démarrage          72 o ← la console, à l’allumage – la mise en route de la console
chaque_minuteur() 124 o ← main() – une fonction fournie par le langage
main()             299 o ← démarrage – ta fonction
VBlank             8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage      28 o ← image() – attend l’image suivante
AttendreVBlank     16 o ← démarrage – attend que l’écran soit libre
AttendreAcces      12 o ← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
AdresseCase        10 o ← poser(), lire(), effacer(), texte() – trouve l’adresse d’une case de l’écran
EffacerCases       12 o ← effacer() – efface des cases
EcrireTexte        10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles       40 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage – vide les deux cartes de l’écran
LireManette        37 o ← bouton() – lit les huit boutons
Tuiles             32 o ← CopierTuiles – 2 lettres sur 43
"."                1 o ← texte() – un texte gravé
"."                1 o ← texte() – un texte gravé
```

Lettres gravées : A.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte()           × 2   36 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()          × 2   30 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
poser()            × 2   26 o sur place, dans main() + AdresseCase (10 o)
bouton()           × 5   25 o sur place, dans main() + LireManette (37 o)
lire()             × 1   11 o sur place, dans main() + AdresseCase (10 o)
image()            × 1    3 o sur place, dans main() + AttendreImage (28 o)
images()           × 1    2 o sur place, dans chaque_minuteur()
```

Tes fonctions appelées :

```
chaque_minuteur() × 2   21 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : le A bouge, le tir garde sa ligne ----
// Le 0.98, et le A bouge avec la croix (comme le 0 du 0.93.1).
// Le tir part de là où est le A... puis il garde SA ligne, même si le A s'en va.

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <lire>       // lit la tuile posée sur une case
#include <effacer>    // efface des cases, ou tout le fond
#include <texte>      // écrit un texte à l’écran
```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;           // le A : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...et sa ligne
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;           // la case d'ARRIVÉE du A, calculée AVANT de bouger (le 0.82)
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. UNE VARIABLE — TX

```
uint8_t tx = 0;           // le tir : sa colonne...
```

7. UNE VARIABLE — TY

```
uint8_t ty = 0;           // NOUVEAU : ...et SA ligne, à lui
```

8. UNE VARIABLE — VOLE

```
uint8_t vole = 0;         // 1 = le tir est en vol ; 0 = pas de tir
```

9. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(ax, ay, ALPHABET[0]);

    while (true) {
        image();

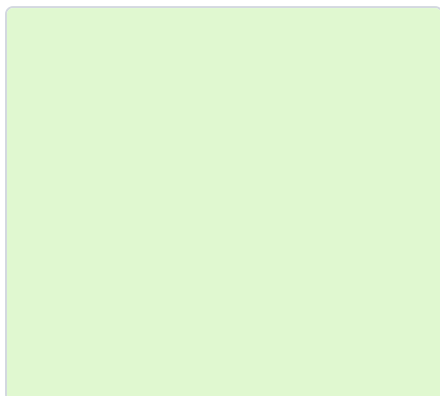
        // ---- le A bouge avec la croix, un pas toutes les 150 ms (le 0.93.1) ----
        if (chaque(150)) {
            nx = ax;                // 1. l'arrivée : on part de la case actuelle...
            ny = ay;
            if (bouton(DROITE)) { // ...et la flèche la change d'une case.
                nx = ax + 1;      // else if : une seule flèche à la fois.
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;      // en colonne 0, 0 - 1 donne 255 (le uint8_t fait le tour)
            } else if (bouton(BAS)) {
                ny = ay + 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
            }
            // 2. l'arrivée est dans l'écran : colonnes 0 à 19, lignes 0 à 16.
            // 255 n'est pas plus petit que 20 : sortir à gauche ou en haut est refusé aussi.
            if (nx < 20 && ny < 17) {
                // 3. ...et c'est une case VIDE (la tuile 0) : le A y va.
                // Le tir n'est pas vide : le A ne marche pas dessus.
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- A lance le tir, depuis la case du A ----
        // ax < 19 : en colonne 19, il n'y a plus de place à droite pour le tir.
        if (bouton(A) && vole == 0 && ax < 19) {
            vole = 1;
            tx = ax + 1;          // juste à droite du A...
            ty = ay;              // NOUVEAU : ...sur la ligne où est le A MAINTENANT.
            texte(tx, ty, ".");  // ty est recopiée UNE fois, au départ du tir.
        }

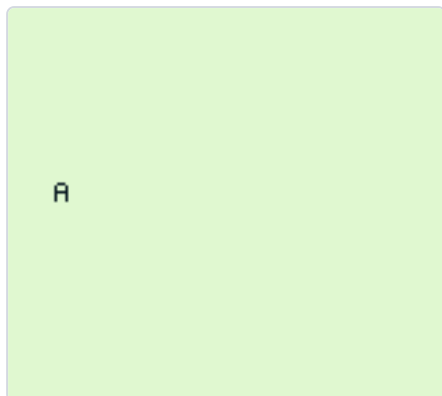
        // ---- le tir avance sur SA ligne, ty ----
        // Si le A monte ou descend pendant le vol, ay change, mais pas ty :
        // le tir continue tout droit, là où il a été lancé.
        if (chaque(100)) {
            if (vole == 1) {
                effacer(tx, ty, 1);
                tx = tx + 1;
                if (tx < 20) {
                    texte(tx, ty, ".");
                } else {
                    vole = 0;
                }
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



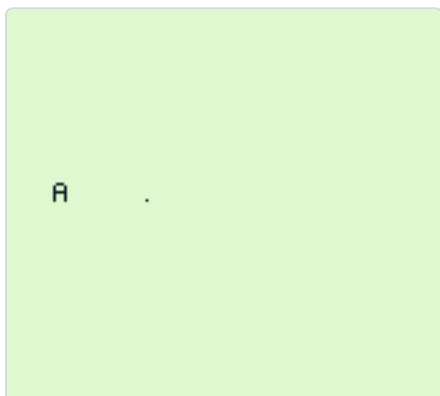
la toute première image — la console vient de démarrer — image 1



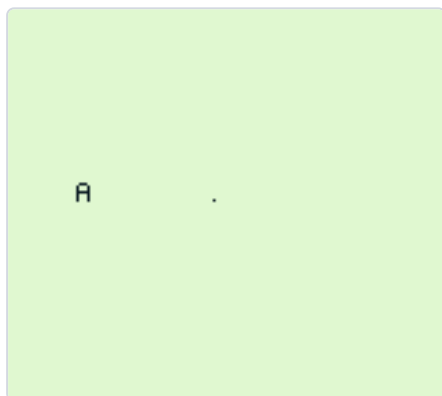
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

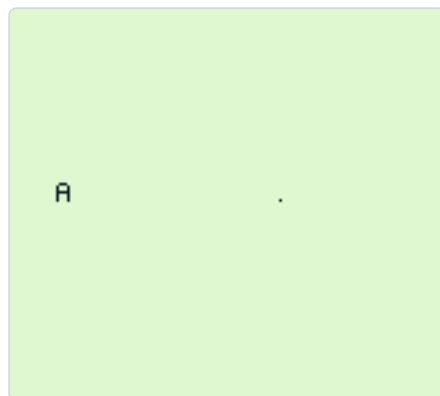
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



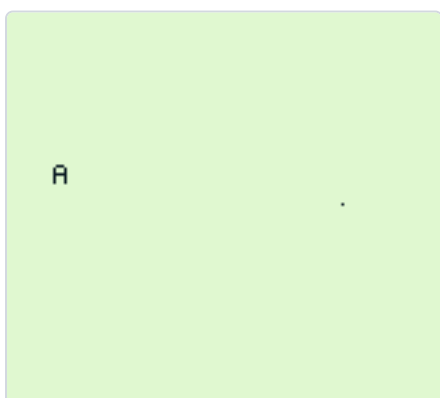
A — l'écran a changé



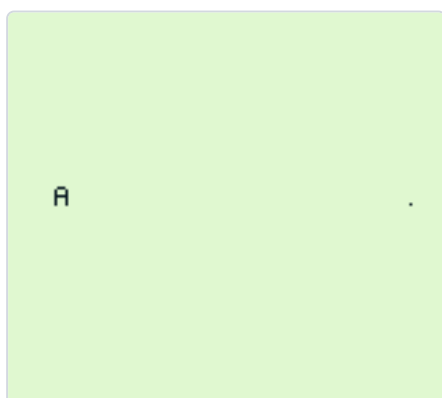
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le A bouge avec la croix. Le bouton A lance un « . » à sa droite ; si le A monte ou descend ensuite, le tir continue sur la ligne où il est parti.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ BAS : le A descend en (2, 11)
 - ✓ A : le tir part à sa droite, ty = 11
 - ✓ le A remonte en ligne 9 : le tir reste sur la ligne 11 (ay = 9, ty = 11)
 - ✓ le tir finit par disparaître au bord
-

Ensuite : **0.99.1. Une lettre qui tire — le A bouge, chaque tir garde sa ligne** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++